

Duty/Standby Pump Lead-Lag Control

Complete RSLogix 500 design for Project 11

Automatic duty rotation, timed lag assist staging, per-pump no-flow and overload protection, reset, and silence.

Design decision

All control logic is written against **internal image bits**, never against **I:** or **O:** addresses directly – and that isolation is now symmetric in both directions. Just as every field input is mapped 1:1 onto a dedicated internal bit, every physical output is driven by exactly one dedicated internal bit, with no combination logic performed at the point the output is written: any OR-ing needed for an indicator (e.g. a fault light that should light on either of two alarm conditions) is resolved *upstream*, in the subroutine that owns that logic, into a single dedicated result bit. A single I/O subroutine (LAD 3) then does nothing but copy bits across the field boundary in both directions – one plain XIC→OTE rung per address, never a branch, never a combination. This isolates the control strategy from the field wiring: I/O can be re-pointed, forced, or simulated in Emulate 500 without touching the duty-rotation, staging, or protection logic. Duty selection and lag staging use ordinary OTE coils with seal-in contacts; no latch or unlatch instructions are used, matching Project 10's convention.

1 Requirements and I/O

Two pumps share duty. Whichever pump is currently **lead** runs alone to satisfy normal demand; the other is **lag** (standby) and is only staged in if demand stays high long enough to indicate the lead pump cannot keep up alone. Lead/lag roles alternate automatically after a runtime interval elapses, so wear is distributed evenly between the two pumps instead of one unit always carrying the duty. Either pump trips independently on loss of flow while commanded to run, or on an overload signal, without disturbing the other pump.

Address	Assigned function	Address	Assigned function
I:0/0	Demand level switch (call for duty pump)	O:0/0	Pump A starter
I:0/1	Assist level switch (call for lag pump)	O:0/1	Pump B starter
I:0/2	Pump A overload contact (N.C., healthy=1)	O:0/2	Pump A fault indicator
I:0/3	Pump B overload contact (N.C., healthy=1)	O:0/3	Pump B fault indicator
I:0/4	Pump A flow switch (flow proven=1)	O:0/4	Common alarm horn
I:0/5	Pump B flow switch (flow proven=1)		
I:0/6	Reset pushbutton		
I:0/7	Silence pushbutton		

1.1 Internal memory

Address	Purpose
B3:3/0--7	Input image: DEMAND, ASSIST_DEMAND, OL_A_HEALTHY, OL_B_HEALTHY, FLOW_A, FLOW_B, RESET_PB, SILENCE_PB (mapped 1:1 from I:0/0-7)
B3:0/0	Duty select (0 = Pump A lead, 1 = Pump B lead)
B3:0/1	CMD_A – Pump A run command; also the dedicated output-image bit for O:0/0
B3:0/2	CMD_B – Pump B run command; also the dedicated output-image bit for O:0/1
B3:0/3	Assist active (lag pump staged in)
B3:0/4	Any-fault summary – dedicated output-image bit for the common horn, O:0/4
B3:0/5	Pump A fault indicator – dedicated output-image bit for O:0/2
B3:0/6	Pump B fault indicator – dedicated output-image bit for O:0/3
B3:1/0, B3:1/1	Pump A no-flow: sealed alarm, silenceable alert
B3:1/2, B3:1/3	Pump B no-flow: sealed alarm, silenceable alert
B3:1/4, B3:1/5	Pump A overload: sealed alarm, silenceable alert
B3:1/6, B3:1/7	Pump B overload: sealed alarm, silenceable alert
B3:2/0--7	OSR storage / one-scan trigger pairs for the four fault events above
B3:2/8, B3:2/9	Duty-rotation trigger bit and its OSR storage bit
T4:0, T4:1	Pump A / Pump B no-flow delay, TON, 5.00 s
T4:2	Lag assist stage-in delay, TON, 10.00 s
T4:3, T4:4	Pump A / Pump B lead-runtime accumulator, RTO, 30.00 s (demo interval)

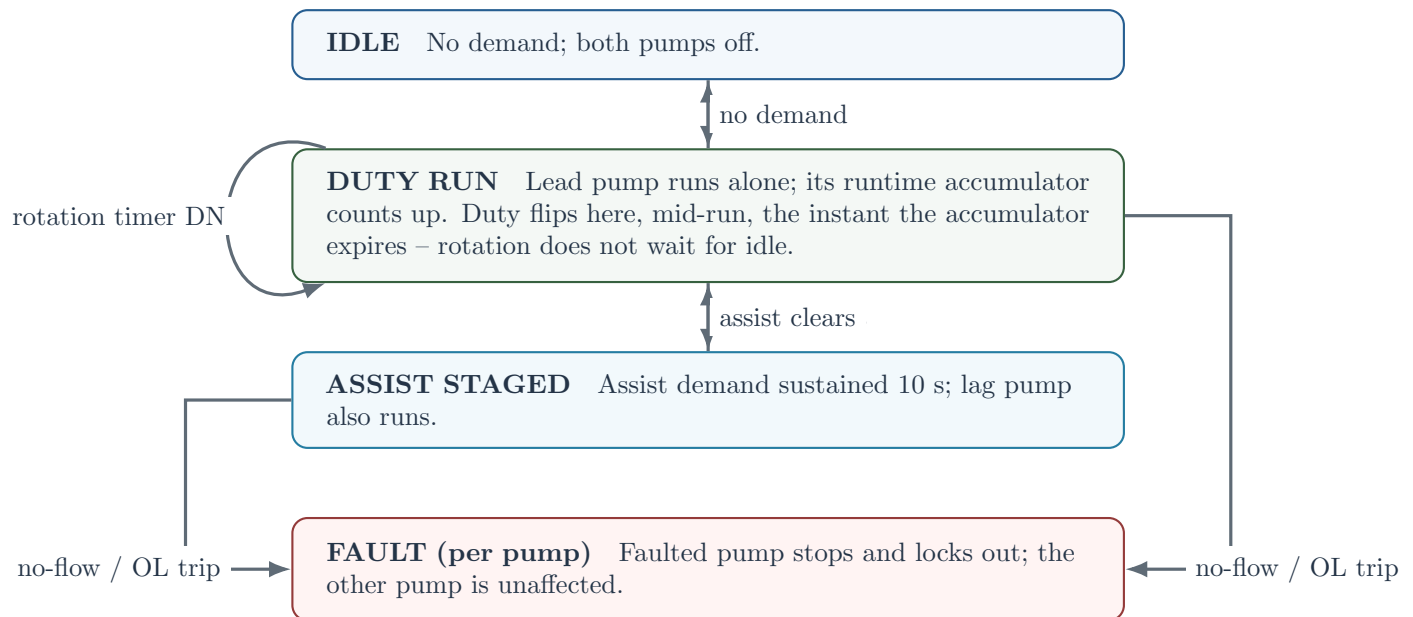
Demo scaling note

A real duty-rotation interval is typically 8–168 running hours. T4:3/T4:4 use a 30-second preset here purely so the rotation, both fault trips, and the assist stage-in can all be exercised within a short Emulate 500 test session. The logic pattern is identical at any preset – only the PRE value changes for a production system.

Dedicated output bits

B3:0/1, B3:0/2, and B3:0/4 are already exactly the value their output should show, so they double directly as output-image bits – no separate copy is needed. B3:0/5 and B3:0/6 exist *only* to be output-image bits: each is the single result of an OR already resolved in LAD 5 (Alarms, below), so that LAD 3 never needs to branch or combine anything – it only ever sees one bit per output.

2 Control Structure

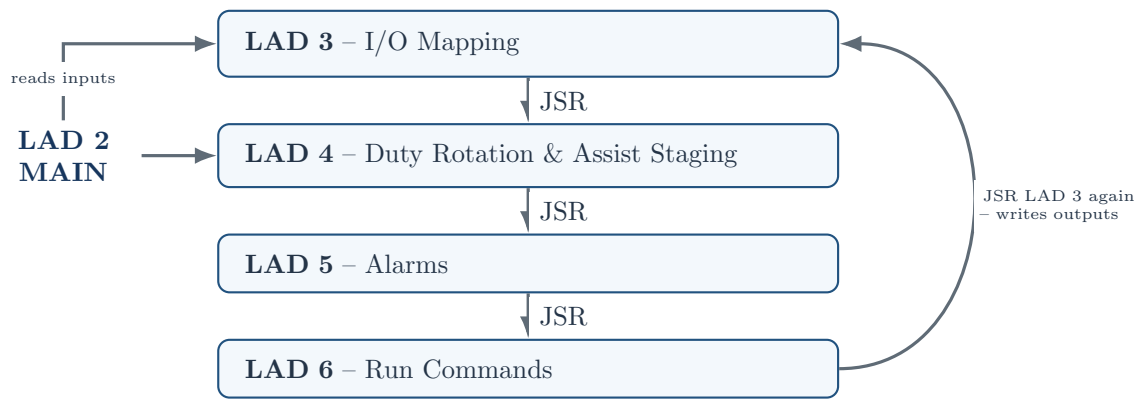


Lead/lag roles are a property of the whole system (B3:0/0), not of an individual pump: whichever pump is currently lead follows the **DUTY RUN** / **ASSIST STAGED** path, and the other follows only the **ASSIST** branch. A fault is scoped to one pump only – it never stops the healthy pump, and it never blocks a duty rotation of the healthy pump on its own schedule.

3 Program Organization

The program is split into a main file and four subroutines, each with a single responsibility. **LAD 2** calls the others in a fixed order every scan, calling the I/O subroutine **twice** – once to refresh inputs before any logic runs, and once more to write outputs after all logic has run.

File	Type	Purpose
LAD 2	MAIN	Calls LAD 3–6 in order, once per scan, calling LAD 3 twice. Contains no control logic of its own.
LAD 3	SBR / RET	I/O Mapping. Maps physical inputs to internal image bits, and dedicated internal image bits to physical outputs. Nothing else – no branching, no combination logic, one plain copy rung per address.
LAD 4	SBR / RET	Duty Rotation. Lead-runtime accumulators, rotation detection, the duty-select toggle, and lag assist stage-in.
LAD 5	SBR / RET	Alarms. Per-pump no-flow and overload delay/trigger/alarm/alert logic, the per-pump fault-indicator bits, and the any-fault summary.
LAD 6	SBR / RET	Run Commands. Combines duty/assist role with alarm state to produce each pump's run command.



Why LAD 3 is called twice, and why that's safe

Inputs must be fresh *before* LAD 4–6 run, but outputs must be written *after* LAD 4–6 have computed the current scan's command and alarm bits – one subroutine cannot be in both places in the call order at once. The fix is to call the same subroutine twice: LAD 3 unconditionally re-maps *all* inputs and *all* outputs every time it runs, so the first call's output-mapping rungs simply re-assert the previous scan's already-known-correct output values (harmless), and the second call's input-mapping rungs simply re-latch input data that hasn't changed since scan start (also harmless, since RSLogix 500 fixes the input image for the whole scan regardless of when in the program an I: address is read). The only rungs that matter each call are the input-mapping rungs on the first call and the output-mapping rungs on the second – and because the physical output image is only written to hardware at the end of the scan, the second call's fresh values are what actually reach the terminals.

Scan-order note

LAD 4's lead-runtime accumulators and LAD 5's no-flow delay timers are gated by each pump's run command (B3:0/1/B3:0/2), but LAD 6 – where those commands are actually written – does not execute until later in the same scan. Both subroutines therefore act on the *previous* scan's run-command value: a one-scan lag of a few milliseconds. This is ordinary subroutine call-order behavior and is negligible next to the 5- and 30-second time constants involved, so no correction is needed. LAD 6 itself always sees current-scan alarm and role data, since LAD 5 and LAD 4 have already run.

4 Complete Ladder Implementation

The following mnemonic rungs are an implementation blueprint, grouped by the file that contains them and renumbered from Rung 000 within each file, matching RSLogix 500 convention. Each timer instruction explicitly shows its time base; with a 0.01-second time base, preset 500 is 5.00 s, 1000 is 10.00 s, and 3000 is 30.00 s. Branches shown in braces are parallel branches. Every stored state uses an ordinary OTE coil with a seal-in contact; no latch or unlatch instructions are used.

LAD 2 – Main Program

Rungs 000–004 – Subroutine calls

Each rung is an unconditional program-control instruction; no permissive contact is needed since a JSR rung with no input condition simply executes every scan, in ladder order. LAD 3 is called twice, deliberately – see the note above.

```
JSR LAD 3 ; I/O Mapping -- refreshes inputs for the logic below
JSR LAD 4 ; Duty Rotation & Assist Staging
JSR LAD 5 ; Alarms
JSR LAD 6 ; Run Commands
JSR LAD 3 ; I/O Mapping -- writes outputs from this scan's results
```

LAD 3 – I/O Mapping

Rungs 000–007 – Input image mapping

Every downstream rung in every other file references these internal bits, never the physical I: addresses.

```
XIC I:0/0 OTE B3:3/0 ; DEMAND
XIC I:0/1 OTE B3:3/1 ; ASSIST_DEMAND
XIC I:0/2 OTE B3:3/2 ; OL_A_HEALTHY
XIC I:0/3 OTE B3:3/3 ; OL_B_HEALTHY
XIC I:0/4 OTE B3:3/4 ; FLOW_A
XIC I:0/5 OTE B3:3/5 ; FLOW_B
XIC I:0/6 OTE B3:3/6 ; RESET_PB
XIC I:0/7 OTE B3:3/7 ; SILENCE_PB
```

Rungs 008–012 – Output image mapping

Every physical output is written from exactly one dedicated internal bit – one plain XIC→OTE rung per address, no branches, no OR logic. Any combination needed to produce that bit (e.g. the two fault-indicator ORs) is already resolved upstream in LAD 5. Ends with an implicit RET.

```
XIC B3:0/1 OTE 0:0/0 ; Pump A starter
XIC B3:0/2 OTE 0:0/1 ; Pump B starter
XIC B3:0/5 OTE 0:0/2 ; Pump A fault indicator
XIC B3:0/6 OTE 0:0/3 ; Pump B fault indicator
XIC B3:0/4 OTE 0:0/4 ; common alarm horn
```

LAD 4 – Duty Rotation & Assist Staging

Rungs 000–001 – Lead-runtime accumulators

Only the pump currently acting as lead, and only while it is actually commanded to run, accumulates toward the next rotation. These retentive timers hold their accumulated value through a pump stop; only the rotation trigger or a manual reset clears them.

```
XIO B3:0/0 XIC B3:0/1 RTO T4:3 Time Base 0.01 PRE 3000 ACC 0 ; A is lead
XIC B3:0/0 XIC B3:0/2 RTO T4:4 Time Base 0.01 PRE 3000 ACC 0 ; B is lead
```

Rung 002 – Detect rotation due, gated to demand present

Rotation is only evaluated while there is demand, so the swap happens as part of normal duty rather than waiting for an idle gap that a continuously-running process might never provide. This does mean the swap can land mid-run: the instant the trigger pulses, the outgoing pump's run command drops and the incoming pump's run command rises on the same scan (LAD 6, Run Commands, below) – a brief bumped transfer, not a hot-standby overlap.

```
{ XIO B3:0/0 XIC T4:3/DN }
{ XIC B3:0/0 XIC T4:4/DN } XIC B3:3/0 OSR B3:2/9 OTE B3:2/8
```

Rung 003 – Duty-select toggle (seal-in flip-flop)

A one-scan pulse on B3:2/8 flips B3:0/0: the first branch sets it from 0 to 1, the second branch holds it at 1 until the next pulse turns it back to 0. This is the same seal-in convention Project 10 uses for mode memory – no OTL/OTU is used.

```
{ XIC B3:2/8 XIO B3:0/0 }
{ XIC B3:0/0 XIO B3:2/8 } OTE B3:0/0
```

Rung 004 – Reset runtime accumulators on rotation

```
XIC B3:2/8 RES T4:3
XIC B3:2/8 RES T4:4
```

Rung 005 – Lag assist stage-in delay

```
XIC B3:3/1 TON T4:2 Time Base 0.01 PRE 1000 ACC 0
```

Rung 006 – Assist-active bit

Seals in once staged; drops automatically when assist demand clears, no reset pushbutton required. Ends with an implicit RET.

```
{ XIC T4:2/DN }
{ XIC B3:0/3 XIC B3:3/1 } OTE B3:0/3
```

LAD 5 – Alarms

Rung 000 – Pump A no-flow delay

```
XIC B3:0/1 XIO B3:3/4 TON T4:0 Time Base 0.01 PRE 500 ACC 0
```

Rung 001 – Pump A no-flow one-scan trigger

```
XIC T4:0/DN OSR B3:2/0 OTE B3:2/1
```

Rungs 002–003 – Pump A no-flow alarm and silenceable alert

```
{ XIC B3:2/1 } OTE B3:1/0  
{ XIC B3:1/0 XIO B3:3/6 }  
  
{ XIC B3:2/1 } OTE B3:1/1  
{ XIC B3:1/1 XIO B3:3/6 XIO B3:3/7 }
```

Rung 004 – Pump B no-flow delay

```
XIC B3:0/2 XIO B3:3/5 TON T4:1 Time Base 0.01 PRE 500 ACC 0
```

Rung 005 – Pump B no-flow one-scan trigger

```
XIC T4:1/DN OSR B3:2/2 OTE B3:2/3
```

Rungs 006–007 – Pump B no-flow alarm and silenceable alert

```
{ XIC B3:2/3 } OTE B3:1/2  
{ XIC B3:1/2 XIO B3:3/6 }  
  
{ XIC B3:2/3 } OTE B3:1/3  
{ XIC B3:1/3 XIO B3:3/6 XIO B3:3/7 }
```

Rungs 008–010 – Pump A overload trip, alarm, and alert

The overload contact is a protective device already, so its loss is treated as an immediate one-scan trigger with no added delay.

```
XIO B3:3/2 OSR B3:2/4 OTE B3:2/5

{ XIC B3:2/5 } OTE B3:1/4
{ XIC B3:1/4 XIO B3:3/6 }

{ XIC B3:2/5 } OTE B3:1/5
{ XIC B3:1/5 XIO B3:3/6 XIO B3:3/7 }
```

Rungs 011–013 – Pump B overload trip, alarm, and alert

```
XIO B3:3/3 OSR B3:2/6 OTE B3:2/7

{ XIC B3:2/7 } OTE B3:1/6
{ XIC B3:1/6 XIO B3:3/6 }

{ XIC B3:2/7 } OTE B3:1/7
{ XIC B3:1/7 XIO B3:3/6 XIO B3:3/7 }
```

Rungs 014–015 – Per-pump fault-indicator bits

Each is the single OR result LAD 3 will copy straight to that pump's fault-indicator output – LAD 3 itself never sees more than one bit per address.

```
{ XIC B3:1/1 }
{ XIC B3:1/5 } OTE B3:0/5 ; Pump A fault indicator

{ XIC B3:1/3 }
{ XIC B3:1/7 } OTE B3:0/6 ; Pump B fault indicator
```

Rung 016 – Any-fault summary

Kept in the same file as the alarm bits it summarizes. Ends with an implicit RET.

```
{ XIC B3:1/0 }
{ XIC B3:1/2 }
{ XIC B3:1/4 }
{ XIC B3:1/6 } OTE B3:0/4
```

LAD 6 – Run Commands

Rung 000 – Pump A run command

Runs as lead on demand, or as lag once staged – either way, locked out by its own sealed faults only.

```
{ XIO B3:0/0 XIC B3:3/0 }
{ XIC B3:0/0 XIC B3:0/3 } XIO B3:1/0 XIO B3:1/4 OTE B3:0/1
```

Rung 001 – Pump B run command

Ends with an implicit RET.

```
{ XIC B3:0/0 XIC B3:3/0 }
{ XIO B3:0/0 XIC B3:0/3 } XIO B3:1/2 XIO B3:1/6 OTE B3:0/2
```

5 How Duty Rotation, Staging, and Protection Work

Duty rotation: only the active lead pump's RT0 accumulates, and only while it is actually running. As soon as it reaches 30.00 s *and* demand is present, a one-scan pulse flips B3:0/0 and resets both accumulators, so the other pump takes over as lead. Because this is gated on demand rather than idle, it can land mid-run: the outgoing pump's run command drops and the incoming pump's run command rises on the same scan (a brief bumped transfer). If the accumulator finishes while the system happens to be idle, the pulse fires the moment demand reappears instead.

Assist staging: the lag pump joins only after ASSIST_DEMAND (a distinct, higher level call) has been present continuously for 10 s – a short spike does not stage the second pump. Assist drops out automatically the instant the high-level call clears; no operator reset is needed for staging itself.

Per-pump protection: each pump has its own no-flow delay and overload trigger, its own sealed alarm, and its own silenceable alert. A fault removes only that pump's run command (LAD 6, Rungs 000/001) – the healthy pump keeps running whether it was duty or assist at the time.

Reset: RESET_PB clears all four alarm bits and re-arms the corresponding delay timer; if the abnormal condition is still present, the delay must expire again before a new trip is sealed.

Silence: SILENCE_PB clears only the alert/indicator bits (and therefore the horn once no other alert is active). The protective alarm bits remain sealed and continue to block that pump until RESET_PB is pressed.

6 Verification Against the Test Criteria

Test action	Expected result
Startup, no demand Assert DEMAND	Both pumps off, B3:0/0=0 (A is lead by default), no faults. Pump A starts (LAD 6, Rung 000); its T4:3 accumulator begins counting.
Hold DEMAND, assert AS- SIST_DEMAND for 10 s	B3:0/3 seals in; Pump B also starts (LAD 6, Rung 001) as lag.
Clear ASSIST_DEMAND	B3:0/3 drops immediately; Pump B stops, Pump A continues alone.

Test action	Expected result
Hold DEMAND continuously past 30 s	B3:0/0 flips to 1 and both accumulators reset the instant T4:3 reaches DN; Pump A's run command drops and Pump B's rises on the same scan, mid-run, without demand ever clearing.
Withhold flow on the running lead pump for 5 s	That pump's no-flow alarm and alert seal in, its run command drops, and its fault indicator energizes; the other pump is unaffected.
Press SILENCE	The alerting pump's indicator and the common horn de-energize; the sealed alarm remains, so the pump stays locked out.
Press RESET with the fault condition cleared	The alarm and alert clear and the delay timer re-arms; the pump is available to run again on its next turn as duty or lag.
Drop OL_x_HEALTHY momentarily	The corresponding overload alarm/alert seal in on the same scan (no delay); reset behaves as above.

7 Implementation Checks

Before testing in Emulate, confirm the following:

1. Every physical input and output address appears in exactly one rung, and only in LAD 3 – all logic in LAD 4, LAD 5, and LAD 6 reads and writes internal B3 bits only.
2. Every output-mapping rung in LAD 3 is a plain XIC→OTE with no branch and no second input instruction; if a rung there ever needs an OR, that is a sign the combination belongs in LAD 5 or LAD 6 instead, feeding a new dedicated bit.
3. LAD 2 contains only the five JSR rungs shown, in order, and no other logic – two of the five target LAD 3.
4. Each alarm/alert/duty/assist bit has exactly one OTE; its own contact provides the seal-in branch.
5. Each OSR uses a unique storage bit, and each trigger bit is written by only one OTE.
6. T4:3 and T4:4 are configured as RTO, not TON – they must hold their accumulated value across a pump stop, and must only be cleared by LAD 4, Rung 004, or a manual RES.
7. T4:0–T4:2 are configured as TON with the 0.01-second time base shown.
8. The duty-select toggle in LAD 4, Rung 003 is driven only by the one-scan pulse B3:2/8; verify in single-scan mode that it changes state exactly once per pulse.
9. Each pump's starter output has only one OTE instruction in the entire program.
10. Confirm the JSR/SBR/RET call chain executes with no missing return: forcing a rung false inside any subroutine should only affect that subroutine's own outputs, never skip the remaining JSR calls in LAD 2.
11. Confirm LAD 3's first-call output writes are harmless: force a command bit true immediately before the second JSR LAD 3 and verify the corresponding physical output reflects that forced value at scan end, not the stale value written on the first call.

Result

The program rotates duty automatically, stages assist only on sustained demand, and isolates each pump's protection from the other – while keeping every rung's logic decoupled from the physical I/O addresses, exactly as requested.